

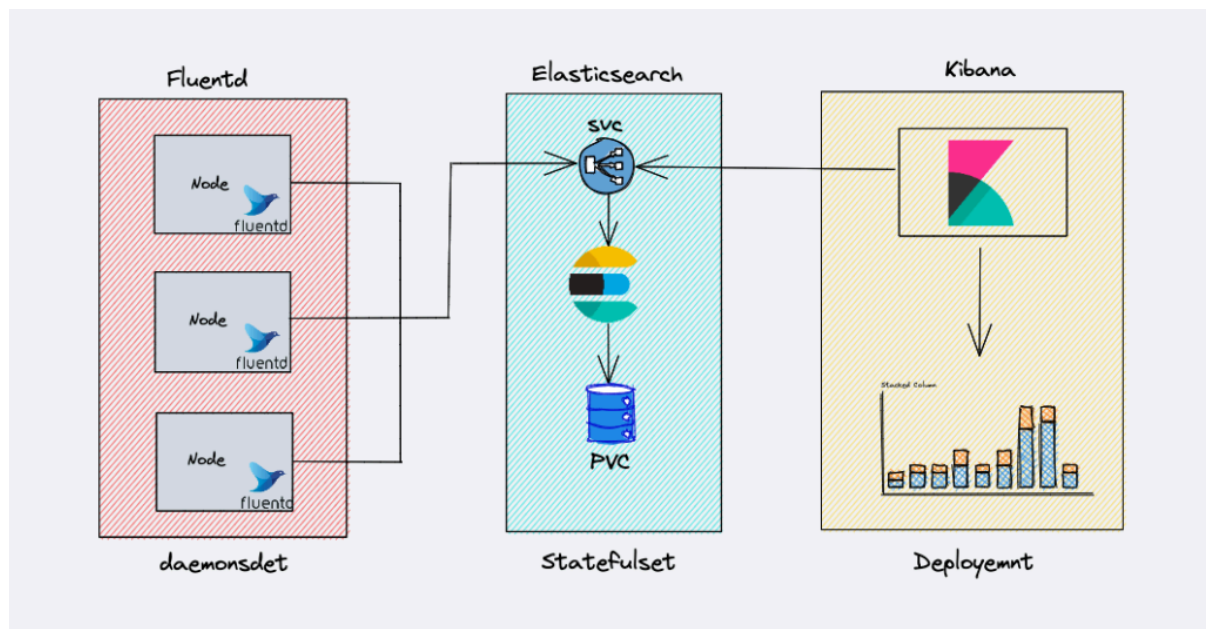
Installation Elasticsearch-Fluentd-kibana

When running multiple services and applications on a Kubernetes cluster, a centralized, cluster-level logging stack can help you quickly sort through and analyze the heavy volume of log data produced by your Pods. One popular centralized logging solution is the **Elasticsearch, Fluentd, and Kibana (EFK)** stack.

Elasticsearch is a real-time, distributed, and scalable search engine which allows for full-text and structured search, as well as analytics. It is commonly used to index and search through large volumes of log data, but can also be used to search many different kinds of documents.

Elasticsearch is commonly deployed alongside **Kibana**, a powerful data visualization frontend and dashboard for Elasticsearch. Kibana allows you to explore your Elasticsearch log data through a web interface, and build dashboards and queries to quickly answer questions and gain insight into your Kubernetes applications.

we'll use **Fluentd** to collect, transform, and ship log data to the Elasticsearch backend. Fluentd is a popular open-source data collector that we'll set up on our Kubernetes nodes to tail container log files, filter and transform the log data, and deliver it to the Elasticsearch cluster, where it will be indexed and stored.



Tasks :

Installation and Configure Elasticsearch

1. EFK Stack Introduction
2. Elasticsearch Introduction
3. Creating a Namespace
4. Creating the Elasticsearch StatefulSet
5. Creating the elasticsearch Service

Installation and Configure Kibana

6. Kibana Introduction
7. Creating the kibana Deployment
8. Creating the kibana Service

Installation and Configure fluentd

9. Fluentd Introduction
10. Creating the Fluentd DaemonSet

Testing Container Logging

7. Creating the container Deployment

Create Dashboard and Visualization

21. Create index pattern
22. Create dashboard for container

STEP 1

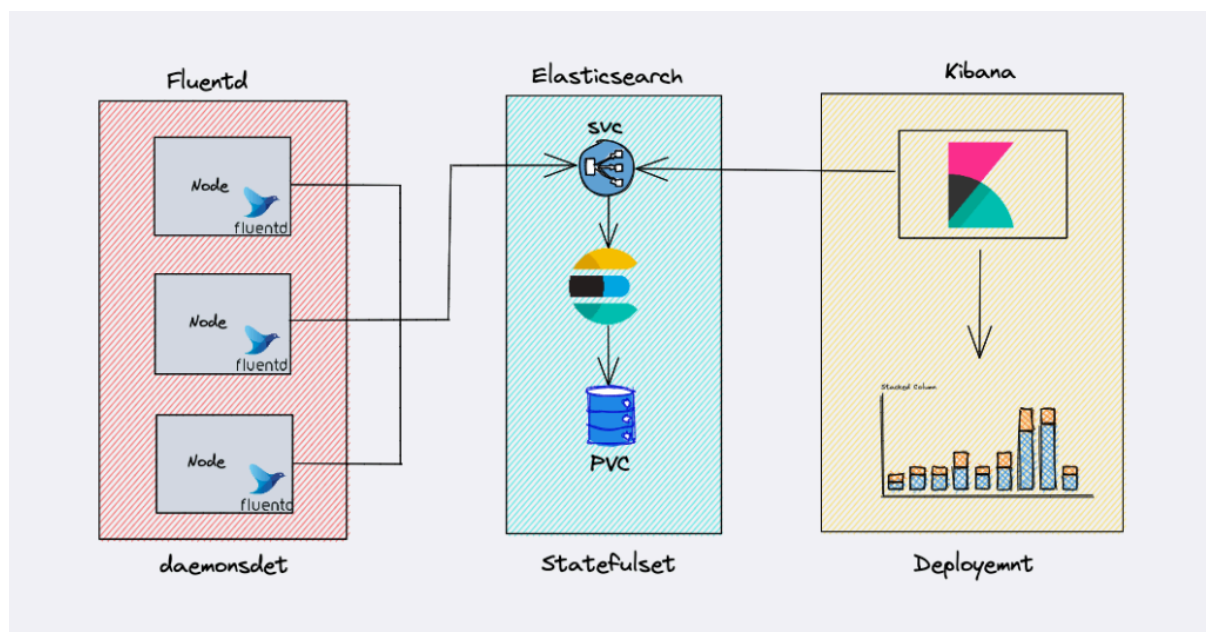
EFK Stack Introduction

When running multiple services and applications on a Kubernetes cluster, a centralized, cluster-level logging stack can help you quickly sort through and analyze the heavy volume of log data produced by your Pods. One popular centralized logging solution is the Elasticsearch, Fluentd, and Kibana (EFK) stack.

Elasticsearch is a real-time, distributed, and scalable search engine which allows for full-text and structured search, as well as analytics. It is commonly used to index and search through large volumes of log data, but can also be used to search many different kinds of documents.

Elasticsearch is commonly deployed alongside **Kibana**, a powerful data visualization frontend and dashboard for Elasticsearch. Kibana allows you to explore your Elasticsearch log data through a web interface, and build dashboards and queries to quickly answer questions and gain insight into your Kubernetes applications.

we'll use **Fluentd** to collect, transform, and ship log data to the Elasticsearch backend. Fluentd is a popular open-source data collector that we'll set up on our Kubernetes nodes to tail container log files, filter and transform the log data, and deliver it to the Elasticsearch cluster, where it will be indexed and stored.

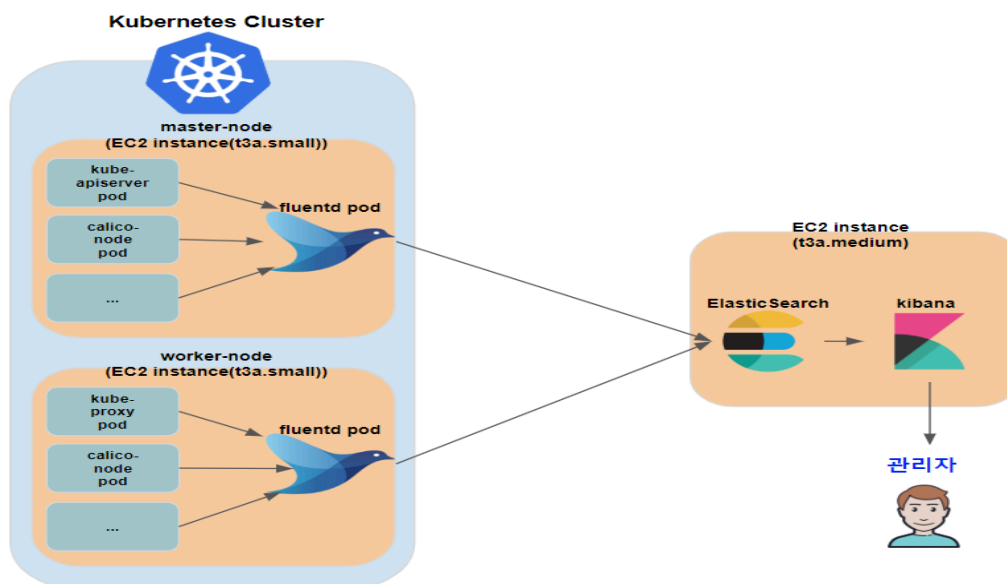


Installation and Configure Elasticsearch

STEP 2

Elasticsearch Introduction

The Elastic Stack — formerly known as the ELK Stack — is a collection of open-source software produced by Elastic which allows you to search, analyze, and visualize logs generated from any source in any format, a practice known as centralized logging. Centralized logging can be useful when attempting to identify problems with your servers or applications as it allows you to search through all of your logs in a single place. It's also useful because it allows you to identify issues that span multiple servers by correlating their logs during a specific time frame.



Elasticsearch is the distributed search and analytics engine at the heart of the Elastic Stack. Logstash and Beats facilitate collecting, aggregating, and enriching your data and storing it in Elasticsearch. Kibana enables you to interactively explore, visualize, and share insights into your data and manage and monitor the stack. Elasticsearch is where the indexing, search, and analysis magic happens.

STEP 3

Creating a kubernetes Namespace

Node : a-kube-master

create a directory for EFK stack :

```
mkdir efk-stack
```

```
cd efk-stack
```

Before we roll out an Elasticsearch cluster, we'll first create a Namespace into which we'll install all of our logging instrumentation. Kubernetes lets you separate objects running in your cluster using a "virtual cluster" abstraction called Namespaces. we'll create a kube-logging

To begin, first investigate the existing Namespaces in your cluster using kubectl:

```
kubectl get namespaces
```

To create the kube-logging Namespace, first open and edit a file called kube-logging.yaml

```
nano kube-logging.yaml
```

Inside your editor, paste the following Namespace object YAML:

```
kind: Namespace
apiVersion: v1
metadata:
  name: kube-logging
```

Once you've created the kube-logging.yaml Namespace object file, create the Namespace using kubectl create with the -f filename flag:

```
kubectl apply -f kube-logging.yaml
```

Output

```
namespace/kube-logging created
```

You can then confirm that the Namespace was successfully created:

```
kubectl get namespaces
```

STEP 4

Creating the Elasticsearch StatefulSet

Node : a-kube-master

Open a file called `elasticsearch_statefulset.yaml` in your favorite editor:

`nano elasticsearch_statefulset.yaml`

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: es-cluster
  namespace: kube-logging
spec:
  serviceName: elasticsearch
  replicas: 1
  selector:
    matchLabels:
      app: elasticsearch
  template:
    metadata:
      labels:
        app: elasticsearch
    spec:
      containers:
        - name: elasticsearch
          image: elasticsearch:7.8.0
          resources:
            limits:
              cpu: 1000m
            requests:
              cpu: 100m
          ports:
            - containerPort: 9200
              name: rest
              protocol: TCP
            - containerPort: 9300
              name: inter-node
              protocol: TCP
          volumeMounts:
            - name: data
              mountPath: /data
      env:
        - name: cluster.name
          value: k8s-logs
        - name: node.name
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
```

```

- name: discovery.seed_hosts
  value: "es-cluster-0.elasticsearch" #es-cluster-1.elasticsearch,es-cluster-2.elasticsearch
- name: cluster.initial_master_nodes
  value: "es-cluster-0" #es-cluster-1,es-cluster-2
- name: ES_JAVA_OPTS
  value: "-Xms512m -Xmx512m"
volumes:
- name: data

initContainers:
- name: fix-permissions
  image: busybox
  command: ["sh", "-c", "chown -R 1000:1000 /data"]
  securityContext:
    privileged: true
  volumeMounts:
- name: data
  mountPath: /data
- name: increase-vm-max-map
  image: busybox
  command: ["sysctl", "-w", "vm.max_map_count=262144"]
  securityContext:
    privileged: true
- name: increase-fd-ulimit
  image: busybox
  command: ["sh", "-c", "ulimit -n 65536"]
  securityContext:
    privileged: true

```

Now, deploy the StatefulSet using kubectl:

```
kubectl apply -f elasticsearch_statefulset.yaml
```

You should see the following output :

Output

statefulset.apps/es-cluster created

STEP 5

Creating the elasticsearch Service

Node : a-kube-master

Open a file called elasticsearch_svc.yaml using your favorite editor:

nano elasticsearch_svc.yaml

We define a Service called elasticsearch in the kube-logging Namespace, and give it the app: elasticsearch label. We then set the .spec.selector to app: elasticsearch so that the Service selects Pods with the app: elasticsearch label.

Paste in the following Kubernetes service YAML:

```
kind: Service
apiVersion: v1
metadata:
  name: elasticsearch
  namespace: kube-logging
labels:
  app: elasticsearch
spec:
  selector:
    app: elasticsearch
  clusterIP: None
  ports:
    - port: 9200
      name: rest
    - port: 9300
      name: inter-node
```

Create the service using kubectl:

kubectl apply -f elasticsearch_svc.yaml

You should see the following output:

Output

service/elasticsearch created .

check service :

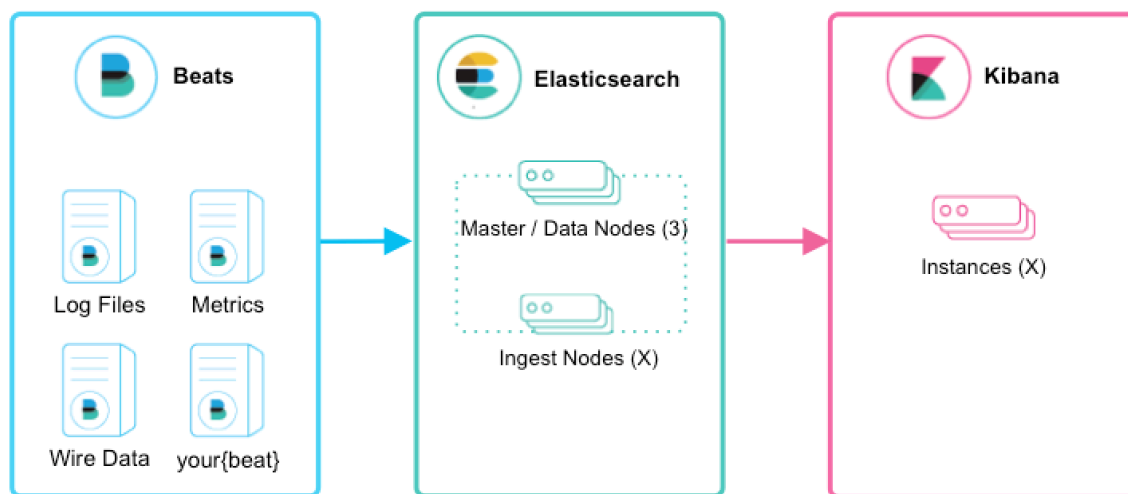
kubectl get services --namespace=kube-logging

Installation and Configure Kibana

STEP 6

Kibana Introduction

Kibana is a free and open frontend application that sits on top of the Elastic Stack, providing search and data visualization capabilities for data indexed in Elasticsearch. Commonly known as the charting tool for the Elastic Stack (previously referred to as the ELK Stack after Elasticsearch, Logstash, and Kibana), Kibana also acts as the user interface for monitoring, managing, and securing an Elastic Stack cluster — as well as the centralized hub for built-in solutions developed on the Elastic Stack.



STEP 7

Creating the kibana Deployment

Node : a-kube-master

Open up a file called kibana-deployment.yaml :

```
nano kibana-deployment.yaml
```

Paste in the following deployment spec:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kibana
  namespace: kube-logging
  labels:
    app: kibana
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kibana
  template:
    metadata:
      labels:
        app: kibana
    spec:
      containers:
        - name: kibana
          image: kibana:7.2.0
          resources:
            limits:
              cpu: 1000m
            requests:
              cpu: 100m
          env:
            - name: ELASTICSEARCH_URL
              value: http://elasticsearch:9200
          ports:
            - containerPort: 5601
```

Now, deploy the kibana using kubectl:

```
kubectl apply -f kibana-deployment.yaml
```

You should see the following output:

Output
deployment.apps/kibana created

Grab the Kibana Pod details using kubectl get:

```
kubectl get pods -n kube-logging
```

STEP 8

Creating the kibana service

Node : a-kube-master

Open up a file called kibana-svc.yaml :

```
nano kibana-svc.yaml
```

Paste in the following service spec:

```
apiVersion: v1
kind: Service
metadata:
  name: kibana
  namespace: kube-logging
spec:
  selector:
    app: kibana
  type: NodePort
  ports:
    - port: 5601
      nodePort: 32000
      targetPort: 5601
```

Now, deploy the kibana using kubectl:

```
kubectl apply -f kibana-svc.yaml
```

Grab the Kibana Pod details using kubectl get:

```
kubectl get svc -n kube-logging
```

STEP 9

Visit the Kibana dashboard

Setting host file :

```
sudo nano /etc/hosts
```

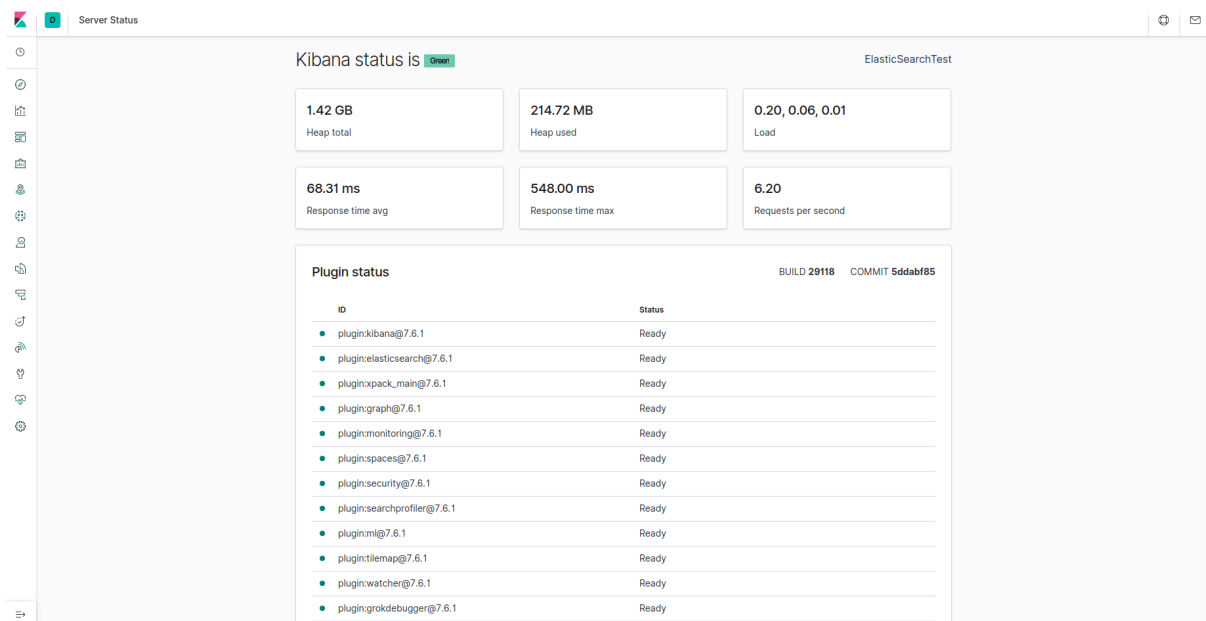
kibana-kube.local a-kube-master IP

kibana-kube.local x.x.5.148

port : 32000

You can check the Kibana server's status page by navigating to the following address :

<http://kibana-kube.local:32000/status>



This status page displays information about the server's resource usage and lists the installed plugins.

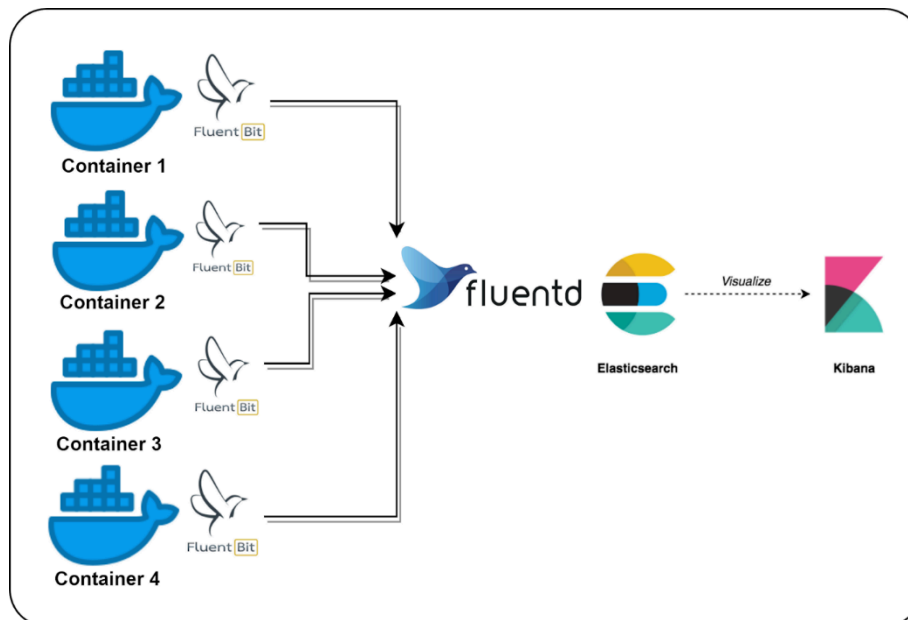
Installation and Configure fluentd

STEP 9

Fluentd Introduction

Node : a-kube-master

Fluentd is an open source data collector, which lets you unify the data collection and consumption for a better use and understanding of data.



we'll use Fluentd to collect, transform, and ship log data to the Elasticsearch backend. Fluentd is a popular open-source data collector that we'll set up on our Kubernetes nodes to tail container log files, filter and transform the log data, and deliver it to the Elasticsearch cluster, where it will be indexed and stored.

STEP 10

Creating the Fluentd DaemonSet

Node : ELK Stack server

Begin by opening a file called fluentd.yaml in your favorite text editor:

```
nano fluentd.yaml
```

Paste in the following configure :

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: fluentd
  namespace: kube-logging
  labels:
    app: fluentd
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: fluentd
  labels:
    app: fluentd
rules:
- apiGroups:
  - ""
  resources:
  - pods
  - namespaces
  verbs:
  - get
  - list
  - watch
---
kind: ClusterRoleBinding
```

apiVersion: rbac.authorization.k8s.io/v1

metadata:

name: fluentd

roleRef:

kind: ClusterRole

name: fluentd

apiGroup: rbac.authorization.k8s.io

subjects:

- kind: ServiceAccount

name: fluentd

namespace: kube-logging

apiVersion: apps/v1

kind: DaemonSet

metadata:

name: fluentd

namespace: kube-logging

labels:

app: fluentd

spec:

selector:

matchLabels:

app: fluentd

template:

metadata:

labels:

app: fluentd

spec:

serviceAccount: fluentd

serviceAccountName: fluentd

tolerations:

- key: node-role.kubernetes.io/master

effect: NoSchedule

containers:

- name: fluentd

image: fluent/fluentd-kubernetes-daemonset:v1.4.2-debian-elasticsearch-1.1

env:

- name: FLUENT_ELASTICSEARCH_HOST

value: "elasticsearch.kube-logging.svc.cluster.local"

```
- name: FLUENT_ELASTICSEARCH_PORT
  value: "9200"

- name: FLUENT_ELASTICSEARCH_SCHEME
  value: "http"

- name: FLUENTD_SYSTEMD_CONF
  value: disable

resources:
  limits:
    memory: 512Mi
  requests:
    cpu: 100m
    memory: 200Mi
  volumeMounts:
    - name: varlog
      mountPath: /var/log
    - name: varlibdockercontainers
      mountPath: /var/lib/docker/containers
      readOnly: true
  terminationGracePeriodSeconds: 30
  volumes:
    - name: varlog
      hostPath:
        path: /var/log
    - name: varlibdockercontainers
      hostPath:
        path: /var/lib/docker/containers
```

Now, roll out the DaemonSet using kubectl :

```
kubectl apply -f fluentd.yaml
```

You should see the following output:

Output

```
serviceaccount/fluentd created
clusterrole.rbac.authorization.k8s.io/fluentd created
clusterrolebinding.rbac.authorization.k8s.io/fluentd created
daemonset.extensions/fluentd created
```

Verify that your DaemonSet rolled out successfully using kubectl:

```
kubectl get ds -n kube-logging
```

```
kubectl get pods -n kube-logging -o wide
```


Testing Container Logging

STEP 15

Creating the container Deployment

Node : a-kube-master

we'll deploy a minimal counter Pod that prints sequential numbers to stdout.

Open up a file called counter.yaml

```
nano counter.yaml
```

Then, paste in the following Pod spec:

```
apiVersion: v1
kind: Pod
metadata:
  name: counter
spec:
  containers:
  - name: count
    image: busybox
    args: [/bin/sh, -c,
          'i=0; while true; do echo "$i: $(date)"; i=$((i+1)); sleep 1; done']
```

This is a minimal Pod called counter that runs a while loop, printing numbers sequentially

Deploy the counter Pod using kubectl:

```
kubectl apply -f counter.yaml
```

Get status :

```
kubectl get pods -o wide
```

Create Dashboard and Visualization

STEP 21

Create index pattern

Open your web browser and access the Kibana web interface using the URL
`http://kibana-kube.local:32000`

create index pattern :

Click the “Stack Management  index patterns  create index pattern ”

Name : logstash-*

Timestamp field : Timestamp

STEP 22

Create dashboard for container

Create Dashboard :

Click the Dashboard  add  Create visualization

Index pattern : logstash-*

Available fields :

1. Timestamp
2. _type
3. Log
4. docker.container_id
5. kubernetes.container_name
6. kubernetes.pod_name

END